

Prueba Técnica — Desarrollador Junior

Proyecto "MesaSitec"

Sitecpro · Plazo de entrega: 1 semana

Bienvenido

Gracias por tu interés en unirse al equipo. Esta prueba no busca que demuestres que sabes todo: busca ver **cómo piensas, cómo lees una especificación y cómo tomas decisiones cuando algo no está dicho explícitamente**.

Vas a construir una versión mínima de un sistema real: una mesa de servicio donde varias organizaciones comparten la misma aplicación. Es pequeño a propósito — cuatro tablas, nueve endpoints, cuatro pantallas— pero tiene reglas de negocio que hay que respetar al detalle.

Lee el enunciado completo antes de escribir la primera línea de código. Buena parte de lo que se evalúa está en los detalles.

1. Reglas del ejercicio

Tema	Regla
Plazo	1 semana desde que recibes este documento
Entrega	Repositorio GitHub con acceso concedido a osanchezm
Uso de IA	Permitido Usa Claude, ChatGPT o lo que prefieras. Solo pedimos que lo declares en DECISIONES.md
Preguntas	Escribe a osanchez@sitecpro.com. Preguntar bien suma puntos;
Propiedad del código	El código es tuyo. No lo usaremos en producción ni lo comercializaremos

Tema	Regla
Requisito eliminatorio	Tu proyecto debe levantar siguiendo tu propio README en menos de 5 minutos . Si no arranca, no podemos evaluarlo

Sobre el uso de IA — léelo con atención

No te vamos a pedir que no uses IA; sería absurdo y además no podríamos verificarlo. Úsala si quieres.

Lo que sí te decimos con transparencia: después de la entrega tendrás una **entrevista técnica** donde vamos a recorrer tu código contigo y te pediremos **implementar un cambio pequeño en vivo**, sobre tu propio proyecto y con nosotros mirando.

No importa si terminas ese cambio. Lo que importa es que sepas dónde tocar. Así que usa la IA para avanzar rápido, pero **entiende lo que entregas**. Si hay una parte que no entendiste del todo, es preferible que lo digas en `DECISIONES.md` a que la descubramos en la entrevista.

Si no te da el tiempo

Prefiimos **menos funcionalidad bien hecha** que todo a medias. Si algo queda fuera, decláralo en el README. La honestidad suma; encontrar omisiones no declaradas resta el doble.

2. El problema

MesaSitec es una mesa de servicio en modalidad SaaS. Varias organizaciones (las llamamos *tenants*) usan la misma instancia de la aplicación y la misma base de datos.

Esta es la regla más importante de toda la prueba: una organización **jamás** debe poder ver ni tocar los datos de otra. Ni por accidente, ni cambiando un ID en la URL, ni por un filtro olvidado en una consulta.

Dentro de cada organización, los usuarios crean solicitudes de soporte. Un agente las va atendiendo siguiendo un flujo de estados. Cada solicitud tiene un plazo de atención (SLA) que el sistema calcula automáticamente.

3. Modelo de datos

Cuatro entidades. Ni una más (salvo que justifiques por qué la necesitas).

Tenant

Campo	Tipo
id	guid
nombre	string
activo	bool

Usuario

Campo	Tipo
id	guid
tenantId	guid
email	string, único a nivel global
passwordHash	string
nombre	string
rol	Admin Agente Solicitante
activo	bool

Categoria

Campo	Tipo
id	guid
tenantId	guid
nombre	string
slaHoras	int
activo	bool

Solicitud

Campo	Tipo
id	guid
tenantId	guid
codigo	string — ver RN-07
titulo	string, entre 5 y 120 caracteres
descripcion	string, entre 10 y 4000 caracteres
categoriaId	guid
prioridad	Baja Media Alta Critica
estado	Nueva Asignada EnProceso Resuelta Cerrada Cancelada
solicitanteId	guid
agenteId	guid, nullable
fechaCreacion	datetime UTC
fechaLimiteSla	datetime UTC, calculado por el servidor
fechaResolucion	datetime UTC, nullable
motivoResolucion	string, nullable
motivoCancelacion	string, nullable

4. Reglas de negocio

RN-01 — Aislamiento entre organizaciones

Todo acceso a datos se filtra por el `tenantId` que viene en el token de sesión. Si un usuario de la organización A pide un recurso de la organización B, la respuesta debe ser **404 Not Found**, no 403 Forbidden.

¿Por qué 404 y no 403? Porque un 403 confirmaría que ese recurso existe. Con un 404 el usuario no puede distinguir entre "no existe" y "existe pero no es tuyo".

RN-02 — Máquina de estados

Una solicitud solo puede moverse por estas transiciones:

Estado actual	Acciones permitidas
Nueva	asignar → Asignada · cancelar → Cancelada
Asignada	iniciar → EnProceso · asignar → Asignada (reasignar a otro agente) · cancelar → Cancelada
EnProceso	resolver → Resuelta · asignar → Asignada · cancelar → Cancelada
Resuelta	cerrar → Cerrada · reabrir → EnProceso
Cerrada	<i>(estado final, no admite acciones)</i>
Cancelada	<i>(estado final, no admite acciones)</i>

Cualquier acción fuera de esta tabla debe responder **409 Conflict** con código: "TRANSICION_INVALIDA".

RN-03 — Permisos por rol

Acción	Admin	Agente	Solicitante
Listar todas las solicitudes de su organización	☒	☒	✗ — solo las que él creó

Acción	Admin	Agente	Solicitante
Ver el detalle de una solicitud	☒	☒	☒ solo las propias
Crear una solicitud	☒	☒	☒
Editar título, descripción, categoría o prioridad	☒	☒	☒ solo las propias y solo si están en estado Nueva
asignar / iniciar / resolver / reabrir	☒	☒	✗
cerrar	☒	☒	☒ solo las propias
cancelar	☒	✗	✗

Si un usuario intenta algo que su rol no permite → **403 Forbidden** con código: "OPERACION_NO_PERMITIDA".

RN-04 — Cálculo del SLA

Cada categoría define un plazo base en horas (slaHoras). La prioridad de la solicitud lo ajusta con un factor:

```
factor = {
  Critica: 0.5,
  Alta:    0.75,
  Media:   1.0,
  Baja:    2.0
}
```

$fechaLimiteSla = fechaCreacion + (categoria.slaHoras \times factor[prioridad]) \text{ horas}$

Ejemplos concretos:

- Categoría Incidente (8 h) con prioridad crítica → límite a las **4 horas** de la creación.
- Categoría Consulta (24 h) con prioridad Baja → límite a las **48 horas** de la creación.

Reglas adicionales:

- El cálculo se hace **siempre en el servidor**. Si el cliente envía fechaLimiteSla en el cuerpo de la petición, se ignora en silencio.

- Si se cambia la prioridad o la categoría de una solicitud que aún no está resuelta, el SLA **se recalcula** — pero `fechaCreacion` no se toca nunca.
- Una solicitud se considera **vencida** si `fechaLimiteSla` ya pasó **y** su estado no es Resuelta, Cerrada ni Cancelada.

RN-05 — Asignación válida

Al ejecutar la acción `asignar`, el `agenteId` recibido debe cumplir todo lo siguiente:

- existe
- está activo
- pertenece a la misma organización
- tiene rol `Agente` o `Admin`

Si falla cualquiera de las cuatro condiciones → **422** con código: `"AGENTE_INVALIDO"`.

RN-06 — Cierre con justificación

- La acción `resolver` exige un motivo de **al menos 20 caracteres**.
- La acción `cancelar` exige un motivo de **al menos 10 caracteres**.

Si no se cumple → **422** con código: `"MOTIVO_REQUERIDO"`.

El motivo se guarda en `motivoResolucion` o `motivoCancelacion` según corresponda. Al resolver, además, se registra `fechaResolucion`.

RN-07 — Código de la solicitud

Formato: `SOL-{año}-{correlativo de 5 dígitos con ceros a la izquierda}`

El correlativo es **independiente por organización y por año**, y empieza en `00001`.

Ejemplo: la cuadragésima segunda solicitud de la Cooperativa Norte en 2026

es `SOL-2026-00042`. La primera del Bufete Sur en 2026 es `SOL-2026-00001`.

No te preocupes por que el correlativo sea infalible ante peticiones simultáneas. Es un problema interesante, pero queda fuera del alcance de esta prueba.

5. Backend

5.1 Tecnologías

Requisito	Detalle
Plataforma	.NET 8 (Web API con controllers o Minimal API, a tu elección). Si prefieres .NET 10, también es válido
ORM	EF Core
Base de datos	SQLite (archivo local — no queremos que instalemos nada)
Migraciones	Se aplican automáticamente al arrancar
Autenticación	JWT propio , HS256, secreto en variable de entorno, expiración 8 horas
Claims mínimos del token	sub, tenantId, rol, email
Contraseñas	BCrypt o PasswordHasher<T> de ASP.NET Core
Documentación	Swagger en /swagger, con el esquema de seguridad Bearer configurado
CORS	Habilitado para http://localhost:5173

Guardar contraseñas en texto plano, con MD5 o con SHA1 desnudo descalifica la entrega. Sin excepciones.

5.2 Organización del código

Queremos ver una separación mínima de responsabilidades: algo como Api / Aplicacion / Dominio / Infraestructura, o el esquema equivalente que prefieras.

Lo que no aceptamos es toda la lógica de negocio metida dentro de los controllers. La máquina de estados, el cálculo del SLA y las validaciones de permisos deben vivir en un lugar donde se puedan probar sin levantar la aplicación completa.

5.3 Manejo de errores

Necesitas un manejador global de excepciones (middleware o `ExceptionHandler`). **Ninguna excepción no controlada debe llegar al cliente como un 500 con stack trace.**

5.4 Pruebas

Mínimo **8 pruebas unitarias con xUnit**, cubriendo:

- la máquina de estados (RN-02)
- el cálculo del SLA (RN-04)
- las reglas de permisos (RN-03)

dotnet test debe pasar en verde.

6. Contrato de la API

Base: `http://localhost:5080/api/v1`

Todo en `application/json`. Fechas en ISO-8601 UTC con sufijo `Z`. Propiedades en `camelCase`.

Este contrato es obligatorio y literal. Vamos a ejecutar pruebas automáticas contra estas rutas, estos nombres de campo y estos códigos de respuesta. Un endpoint que funcione pero devuelva `fecha_creacion` en lugar de `fechaCreacion` cuenta como no implementado.

6.1 Formato de error

Todas las respuestas de error (4xx y 5xx) usan `Content-Type: application/problem+json` con esta forma:

```
{
  "type": "https://mesasitec.local/errores/transicion-invalida",
  "title": "Transición inválida",
  "status": 409,
  "detail": "No se puede aplicar 'resolver' sobre una solicitud en estado 'Nueva'.",
  "codigo": "TRANSICION_INVALIDA",
  "errores": {
    "titulo": ["El título debe tener al menos 5 caracteres."]
  }
}
```

- `codigo` es **obligatorio en todos los errores**. Es el campo que revisan las pruebas automáticas.
- `errores` aparece solo en errores de validación (400 y 422).

Códigos que debes usar:

Situación	HTTP	codigo
Token ausente, inválido o expirado	401	NO_AUTENTICADO
Credenciales incorrectas en el login	401	NO_AUTENTICADO
El rol no permite la operación	403	OPERACION_NO_PERMITIDA
Recurso inexistente o de otra organización	404	RECURSO_NO_ENCONTRADO
Transición de estado no permitida	409	TRANSICION_INVALIDA
Agente inválido al asignar	422	AGENTE_INVALIDO
Falta el motivo o es muy corto	422	MOTIVO_REQUERIDO
Parámetro de consulta fuera de rango	400	PARAMETRO_INVALIDO
Error de validación de campos	422	VALIDACION

6.2 Endpoints

#	Método	Ruta	Descripción	Éxito
1	POST	/auth/login	Autenticación	200
2	GET	/me	Perfil del usuario del token	200
3	GET	/categorias	Categorías activas de su organización	200
4	GET	/solicitudes	Listado paginado y filtrado	200
5	POST	/solicitudes	Crear solicitud	201
6	GET	/solicitudes/{id}	Detalle	200
7	PUT	/solicitudes/{id}	Editar solicitud	200
8	POST	/solicitudes/{id}/transiciones	Ejecutar una acción del flujo	200
9	GET	/health	Estado del servicio — sin autenticación	200

1 • POST /auth/login

// petición

```
{ "email": "agente1@norte.test", "password": "Sitec.2026" }
```

// respuesta 200

```
{
  "accessToken": "eyJhbGciOi...",
  "expiraEn": 28800,
  "usuario": {
    "id": "...", "nombre": "...", "email": "...",
    "rol": "Agente", "tenantId": "...", "tenantNombre": "Cooperativa Norte"
  }
}
```

2 • GET /me

Devuelve el mismo objeto usuario de arriba.

3 • GET /categorias

```
[ { "id": "...", "nombre": "Incidente", "slaHoras": 8 } ]
```

Solo categorías activas y solo de la organización del token.

4 • GET /solicitudes

Parámetros de consulta:

Parámetro	Tipo	Notas
estado	enum	filtro exacto
prioridad	enum	filtro exacto
categoriaId	guid	filtro exacto
agenteId	guid	filtro exacto
q	string	busca en titulo, descripcion y codigo, sin distinguir mayúsculas
vencidas	bool	ver definición en RN-04
page	int	default 1
pageSize	int	default 20, máximo 100
sort	string	fechaCreacion, -fechaCreacion, prioridad, -prioridad, codigo. Default -fechaCreacion

Si `pageSize > 100` o `page < 1` → **400** CON codigo: "PARAMETRO_INVALIDO".

Al ordenar por prioridad, el orden es **semántico** (Critica > Alta > Media > Baja), no alfabético.

El filtrado, la búsqueda, el ordenamiento y la paginación se resuelven en el servidor. Traer todos los registros y filtrarlos en el navegador cuenta como no implementado.

```
// respuesta 200
{
  "items": [
    {
      "id": "...",
      "codigo": "SOL-2026-00001",
      "titulo": "No puedo acceder al portal",
      "estado": "Nueva",
      "prioridad": "Alta",
      "categoria": { "id": "...", "nombre": "Incidente" },
      "agente": null,
      "fechaCreacion": "2026-01-15T08:00:00Z",
      "fechaLimiteSla": "2026-01-15T14:00:00Z",
      "vencida": false
    }
  ],
  "page": 1,
  "pageSize": 20,
  "total": 47,
  "totalPaginas": 3
}
```

Cuando hay agente asignado, el campo agente es { "id": "...", "nombre": "..." }.

5 • POST /solicitudes

```
// petición
{
  "titulo": "No puedo acceder al portal",
  "descripcion": "Al ingresar mis credenciales el sistema me devuelve a la pantalla de login.",
  "categoriaId": "...",
  "prioridad": "Alta"
}
```

Responde **201** con el objeto completo de la solicitud y la cabecera Location apuntando al nuevo recurso.

6 • GET /solicitudes/{id}

Devuelve el objeto completo, incluyendo descripcion, solicitante, motivoResolucion, motivoCancelacion y fechaResolucion.

7 • PUT /solicitudes/{id}

```
{ "titulo": "...", "descripcion": "...", "categoriaId": "...", "prioridad": "Critica" }
```

Recuerda RN-03 (un Solicitante solo edita las propias y solo en estado Nueva) y RN-04 (cambiar prioridad o categoría recalcula el SLA).

8 • POST /solicitudes/{id}/transiciones

```
// asignar
{ "accion": "asignar", "agenteId": "..." }

// resolver
{ "accion": "resolver", "motivo": "Se restableció la contraseña del usuario y se validó el acceso." }

// iniciar, cerrar, reabrir
{ "accion": "iniciar" }

// cancelar
{ "accion": "cancelar", "motivo": "Duplicada de SOL-2026-00012." }
```

Acciones válidas: asignar, iniciar, resolver, cerrar, reabrir, cancelar.

Responde **200** con la solicitud actualizada.

9 • GET /health

```
{ "estado": "ok" }
```

Sin autenticación.

6.3 Datos semilla (obligatorios)

Al arrancar, si la base de datos está vacía, la aplicación debe sembrarla automáticamente.

Usa la variable de entorno SEED_FECHA_BASE (valor por defecto 2026-01-15T08:00:00Z) como punto de referencia temporal: **todas las fechas de los datos semilla deben generarse como desplazamientos fijos respecto a ella**, nunca respecto a DateTime.UtcNow. Así los datos son idénticos sin importar cuándo se ejecute.

Organizaciones: Cooperativa Norte y Bufete Sur

Contraseña de todos los usuarios semilla: Sitec.2026

Email	Organización	Rol
admin@norte.test	Cooperativa Norte	Admin
agente1@norte.test	Cooperativa Norte	Agente
agente2@norte.test	Cooperativa Norte	Agente
user1@norte.test	Cooperativa Norte	Solicitante
user2@norte.test	Cooperativa Norte	Solicitante
admin@sur.test	Bufete Sur	Admin

Email	Organización	Rol
user1@sur.test	Bufete Sur	Solicitante

Categorías (crear en ambas organizaciones):

Nombre	slaHoras
Incidente	8
Requerimiento	40
Consulta	24
Falla crítica	4

Solicitudes: 25 en Cooperativa Norte y 8 en Bufete Sur, repartidas entre todos los estados y todas las prioridades. Al menos 5 vencidas y al menos 3 resueltas en Cooperativa Norte.

7. Frontend

7.1 Tecnologías

Requisito	Detalle
Framework	Vue 3 con <code><script setup></code>
Lenguaje	TypeScript en modo strict
Build	Vite , corriendo en el puerto 5173
Ruteo	Vue Router , con guard para las rutas privadas
Estado	Pinia

Requisitos adicionales:

- `tsc --noEmit` debe pasar **sin errores**.

- **Prohibido el any explícito.** Si en algún punto es inevitable, justifícalo en DECISIONES.md.
- Los DTOs de la API deben estar tipados en TypeScript. Puedes escribirlos a mano o generarlos desde el OpenAPI (generarlos suma puntos).
- Un **único módulo cliente HTTP** centralizado, que inyecte el token en cada petición y redirija a /login ante un 401.

7.2 Sobre el diseño visual

No evaluamos el diseño gráfico. Usa Tailwind, PrimeVue, Vuetify, CSS propio o lo que te resulte más rápido.

Lo que sí evaluamos es que cada vista maneje sus tres estados: **cargando**, **vacío** y **error**. Una tabla que se queda en blanco mientras carga, sin ninguna señal, cuenta como incompleta.

7.3 Vistas requeridas

Ruta	Qué debe contener
/login	Formulario de acceso con manejo de credenciales inválidas
/solicitudes	Tabla con filtros, búsqueda y paginación (todo server-side)
/solicitudes/nueva	Formulario de creación con validación en el cliente
/solicitudes/:id	Detalle, con los botones de acción disponibles según estado y rol
/solicitudes/:id/editar	Reutiliza el mismo componente de formulario en modo edición

7.4 Atributos `data-testid` (obligatorio)

Vamos a ejecutar pruebas automáticas de interfaz contra estos selectores. **Un data-testid faltante o mal escrito equivale a la funcionalidad no entregada**, aunque la pantalla funcione perfectamente al usarla a mano. Cópialos literalmente.

Global

```
app-nav
nav-usuario-nombre
```

nav-usuario-rol
btn-logout
toast-mensaje

Login

login-email
login-password
login-submit
login-error

Listado de solicitudes

btn-nueva-solicitud
filtro-estado
filtro-prioridad
filtro-categoria
filtro-vencidas
filtro-busqueda
btn-limpiar-filtros
tabla-solicitudes
fila-solicitud (uno por fila, y además con el atributo data-codigo="SOL-2026-00001")
celda-codigo
celda-estado
celda-prioridad
celda-sla
badge-vencida
paginacion-anterior
paginacion-siguiente
paginacion-info
listado-vacio
listado-cargando

paginacion-info debe contener exactamente el formato: Página X de Y – Z resultados

Formulario (el mismo componente sirve para crear y editar)

form-titulo
form-descripcion
form-categoria
form-prioridad
form-submit
form-cancelar
error-titulo
error-descripcion
error-categoria

Detalle

detalle-codigo
detalle-titulo
detalle-descripcion
detalle-estado

```
detalle-prioridad
detalle-categoria
detalle-agente
detalle-fecha-creacion
detalle-fecha-limite
detalle-vencida
detalle-motivo
btn-editar
btn-accion-asignar
btn-accion-iniciar
btn-accion-resolver
btn-accion-cerrar
btn-accion-reabrir
btn-accion-cancelar
modal-accion
modal-select-agente
modal-motivo
modal-error
modal-confirmar
modal-cancelar
```

7.5 Regla de visibilidad de los botones de acción

Los botones `btn-accion-*` que **no** correspondan al estado actual de la solicitud o al rol del usuario **no deben existir en el DOM**.

No basta con deshabilitarlos ni con ocultarlos por CSS: las pruebas verifican que el elemento **no esté presente**.

Ejemplo: un usuario con rol solicitante viendo una solicitud en estado Nueva debe ver únicamente `btn-editar` y `btn-accion-cerrar`... y ni siquiera `btn-accion-cerrar`, porque Nueva no admite cerrar. Revisa RN-02 y RN-03 juntas.

8. Qué debes entregar

8.1 El proyecto debe arrancar (requisito eliminatorio)

Tu `README.md` debe permitir levantar todo en **máximo 4 comandos y menos de 5 minutos**, dejando funcionando:

- API en `http://localhost:5080` — con `/health` respondiendo y `/swagger` accesible
- Frontend en `http://localhost:5173`
- Base de datos migrada y sembrada, sin pasos manuales

Un docker-compose.yml que reemplace todo por docker compose up -d --build es **opcional** y suma puntos.

8.2 Estructura del repositorio

```
/
├── README.md
├── DECISIONES.md
├── .env.example
├── backend/
│   ├── src/{Api,Aplicacion,Dominio,Infraestructura}
│   └── tests/
└── frontend/
    └── src/{api,components,views,stores,types,router}
```

8.3 README.md

- Requisitos previos (versiones de .NET, Node, etc.)
- Cómo levantar el proyecto, paso a paso
- Credenciales de prueba
- **Qué está implementado y qué no.** Si algo quedó fuera, dilo aquí

8.4 DECISIONES.md (máximo 1 página)

Este documento pesa en la evaluación tanto como el código. Queremos leer:

1. **Tres decisiones técnicas** que tomaste, con la alternativa que descartaste y por qué.
2. **Qué hiciste con ayuda de IA y qué escribiste a mano.**
3. Qué harías distinto si tuvieras una semana más.
4. **El punto donde te atascaste y cómo lo resolviste.** No hay respuesta incorrecta aquí; "no me atasqué en nada" sí es una mala respuesta.

8.5 Higiene del repositorio

- Mínimo **8 commits** con mensajes significativos. Un único commit "initial commit" con todo el proyecto no nos deja ver cómo trabajas.
 - Sin secretos versionados. Usa .env.example con valores de ejemplo.
 - .gitignore correcto: nada de bin/, obj/, node_modules/, ni el archivo .db.
-

9. Sugerencia de orden de trabajo

No es obligatorio seguir este orden, pero si no sabes por dónde empezar, esta ruta reduce el riesgo de quedarte sin tiempo:

1. **Modelo de datos + EF Core + semilla.** Verifica con Swagger que puedes leer datos antes de seguir.
2. **Login con JWT y el endpoint /me.** Con esto ya tienes la base de todo el resto.
3. **GET /solicitudes con el filtro por tenant.** Aquí implementas RN-01, que es la regla más importante. Hazla bien desde el principio: si la dejas para el final, tendrás que tocar todas las consultas.
4. **Máquina de estados y cálculo del SLA en el dominio,** con sus pruebas unitarias. Es la parte más lógica y la más fácil de probar sin interfaz.
5. **El resto de los endpoints.**
6. **Frontend,** en este orden: login → listado → detalle → formulario.
7. **README, DECISIONES y limpieza del repositorio.** Reserva tiempo real para esto; no lo dejes para los últimos veinte minutos.

10. Preguntas frecuentes

¿Puedo usar una plantilla o un starter kit? Sí, siempre que lo declares en `DECISIONES.md`.

¿Puedo usar una librería para la máquina de estados? Preferimos que la implementes tú. Es una de las partes que más nos dice sobre cómo modelas.

¿Puedo cambiar SQLite por otra base de datos? No, para esta prueba no. Necesitamos que arranque sin instalar nada.

¿Y si no termino el frontend? Entrega lo que tengas y decláralo en el README. Un backend sólido con dos pantallas terminadas vale más que todo a medias.

¿Puedo agregar funcionalidad que no está en el enunciado? Solo si lo esencial ya está completo. Y decláralo. No agregues entidades al modelo sin justificarlo.

¿Qué pasa si encuentro una ambigüedad en el enunciado? Pregunta. Si prefieres no esperar, toma una decisión, impleméntala y explícala en `DECISIONES.md`. Ambas rutas son válidas; quedarte paralizado no.

11. Checklist antes de entregar

- ☐ El proyecto levanta siguiendo mi propio README, desde cero, en menos de 5 minutos
- ☐ GET /health responde 200 sin token
- ☐ /swagger está accesible y documenta los 9 endpoints
- ☐ Los datos semilla se crean solos y las credenciales del README funcionan
- ☐ Probé iniciar sesión con user1@sur.test e intentar abrir una solicitud de Cooperativa Norte → recibo 404
- ☐ Todos los data-testid de la sección 7.4 existen en el DOM, escritos exactamente igual
- ☐ Los botones de acción no permitidos **no se renderizan** (no solo deshabilitados)
- ☐ paginacion-info usa el formato exacto Página X de Y – Z resultados
- ☐ tsc --noEmit pasa sin errores y no hay any explícito sin justificar
- ☐ dotnet test pasa con al menos 8 pruebas
- ☐ No hay secretos, bin/, obj/, node_modules/ ni .db versionados
- ☐ README.md y DECISIONES.md están completos y son honestos
- ☐ Hay al menos 8 commits con mensajes significativos
- ☐ Concedí acceso al repositorio a osanchezm en GitHub

Cualquier duda, escribenos a osanchez@sitecpro.com. Mucho éxito.